MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)
SEM 2 2025/2026

LAB 08:
HUSKYLENS AI VISION AND REAL TIME COLOR DETECTION

SECTION 1
GROUP 10

INSTRUCTOR:
DR WAHJU SEDIONO

Name	Matric Number
NUR IRDINA BINTI OMAR	2312378
MUHAMMAD IRSYAD ILHAM BIN AZIZAN	2217555
NURIN NAJWA BINTI KAMARUZAMAN	2416554

DATE OF SUBMISSION: WEDNESDAY, 6 MAY 2026

Abstract

This experiment was conducted to investigate the integration of the HuskyLens AI Vision Sensor with an Arduino Uno to perform real-time object and color detection. The main objective was to establish UART serial communication between both HuskyLens and Arduino, train the sensor to recognize specific objects and colours, and control hardware output like LED and RGB LED based on the detected ID. Two experiments were conducted on this integration which object recognition control for Task 1 and integrated RGB color display for Task 2. For Task 1, HuskyLens was trained to detect the trained object and turn ON the LED when object detected while for Task 2, it was trained to recognize multiple colours and display matching colors on the RGB LED. The LED and RGB LED were controlled by the detected ID sent by HuskyLens to Arduino. This demonstrated that the system could effectively perform AI-based visual detection and hardware automation. Basically, this experiment highlights the practical use of embedded AI vision systems in automation, smart sensing and interactive mechatronic applications.

Table of Contents

Abstract **2**
Table of Contents **3**
Introduction **4**
Task 1 - Object Recognition Control **5**
2. Material and Equipments 5
3. Experimental Setup 5
4. Methodology 6
5. Data Collection 8
6. Data Analysis 8
7. Results 9
Task 2- Multi-Color Recognition **10**
2. Material and Equipment 10
3. Experimental Setup 10
4. Methodology 11
5. Data Collection 14
6. Results 14
7. Data Analysis 15
Discussion **15**
Conclusion **16**
Recommendation **16**
References **16**
Appendices **17**
Acknowledgement **22**
Student's Declaration **23**

Introduction

Artificial Intelligence vision systems are widely used in modern automation, robotics, and smart electronics because they allow machines to identify visual patterns and make decisions automatically. Meanwhile, HuskyLens is a smart AI camera with built-in machine learning functions that can detect objects, recognize colors and track targets without requiring complex AI programming. Arduino acts as a microcontroller where it receives visual recognition data from HuskyLens in the form of IDs, processes it and controls the output. This demonstrates the practical application of embedded systems, where sensors act as input devices and microcontrollers act as processing units for automated control systems.

This experiment was conducted to explore the integration of the HuskyLens AI Vision Sensor with an Arduino Uno for real-time object and color recognition applications. The main purpose was to understand how an AI-based vision sensor can communicate with a microcontroller through UART serial communication and how the transmitted data can be used to control hardware outputs such as LEDs and RGB LEDs. The objective of this experiment was to establish UART serial communication between HuskyLens and Arduino, train HuskyLens to recognize objects and colours, interpret the detected IDs through Arduino and control the output based on the sensor's visual input.

This experiment covered object detection and multi-color recognition tasks to evaluate the sensor's ability to identify trained targets accurately. The HuskyLens should detect predefined objects and colors, transmit the correct IDs to Arduino, and activate the intended LED outputs with high accuracy under proper environmental conditions. However, environmental factors like ambient lighting and shadows also training quality will influence the detection accuracy. So, a controlled environment will improve reliability and consistency of the results.

Task 1 - Object Recognition Control

In this task, a standard LED was connected to Digital Pin 13. LED turned ON when “ Object 1 “ detected , and LED turned OFF when the camera saw nothing.

2. Material and Equipments

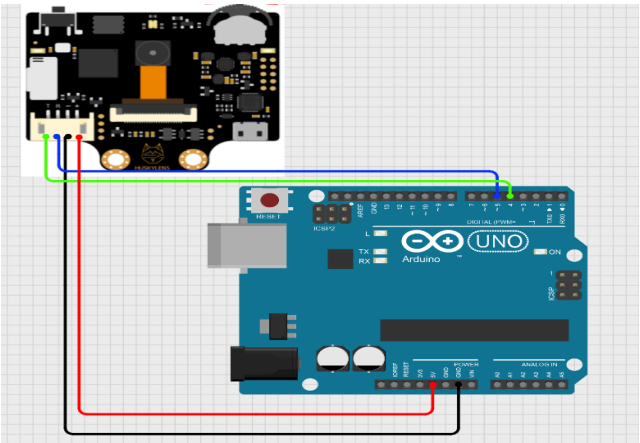
Cytron CT Uno
USB Cable
Gravity HuskyLens AI Vision Camera
1 Led
1 220 ohms resistors
Jumper wires
Breadboard

3. Experimental Setup

-Husky Lenz Connections :

- VCC to Arduino 5V
- GND to Arduino GND
- TX to Arduino D4 (SoftwareSerial RX)
- RX to Arduino D5 (SoftwareSerial TX)

-LED Pin Connection : LED Pin to Arduino D13



Circuit diagram connection for object recognition control

4. Methodology

Hardware Setup

First, all components were connected properly. The HuskyLens camera was interfaced with the Arduino Uno using UART communication. The VCC and GND pins were connected to the Arduino's 5V and GND, while the TX and RX pins were connected to digital pins 4 and 5 respectively using SoftwareSerial. In addition, an LED was connected to digital pin 13 with a 220Ω resistor to ensure safe current flow.

HuskyLens Test

HuskyLens was tested manually. The device settings were accessed using the onboard control knob and it was set to serial 9600 to enable communication with the Arduino.

Object Recognisation

After that, the HuskyLens was set to Object Recognisation mode. A single object such as a mobile phone was trained and assigned as object ID1.

Programming and Implementation

To read the detection data from the camera, the Arduino was programmed using the HuskyLens library. The LED turned ON when the camera detects object ID1.Otherwise, the LED remains off. From this, it can be seen that LED can act as an indicator of successful object detection.

Arduino Coding Code :

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

const int ledPin = 13;

void setup() {
  Serial.begin(115200);
  mySerial.begin(9600);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  while (!huskylens.begin(mySerial)) {
```

```
    Serial.println(F("Begin failed! Check wiring and Protocol Serial 9600"));
    delay(500);
  }

  Serial.println("HuskyLens Ready!");
}

void loop() {
  bool objectDetected = false;

  if (!huskylens.request()) {
    digitalWrite(ledPin, LOW);
    return;
  }

  while (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    if (result.command == COMMAND_RETURN_BLOCK) {
      Serial.print("Object ");
      Serial.print(result.ID);
      Serial.println(" detected!");

      if (result.ID == 1) {
        objectDetected = true;
      }
    }
  }

  if (objectDetected == true) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Testing

-The LED turned ON when the trained object was detected.

-The LED turned OFF when no trained object was detected.

5. Data Collection

During the experiment , several trials have been tried by placing the smartphone in the front of the camera and showing the LED response when connected to Arduino Digital Pin 13.
In this experiment, the HuskyLens was trained to recognize one specific object like a mobile phone using the Object Recognition Mode. The trained object was assigned as Object 1 in the HuskyLens system. When Husky Lens successfully detected the smartphone, the Arduino activated the LED. When the smartphone was removed from the camera view, the LED turned OFF automatically. For every trial, the trained object was shown in front of the HuskyLens camera under normal lighting while the LED responses were observed and recorded.

Trial	Object Presented	Detected ID	LED Status
1	Smartphone	ID 1	ON
2	No Object	No Detection	OFF
3	Smartphone	ID 1	ON
4	Smartphone	ID 1	ON

6. Data Analysis

Based on the experiment , the HuskyLens was able to identify the trained object during most trials. When the HuskyLens camera recognized Object 1, the Arduino received the detection data via serial communication and turned ON the LED that was attached to Pin 13 whenever the camera identified Object 1.

The LED only turned ON when the correct trained object was identified. The LED remained OFF during trials with no object or an untrained object because the HuskyLens failed to provide a valid detection ID. This shows that the object recognition system operated correctly according to the programmed conditions.

7. Results

Based on the results obtained, the system was able to perform object recognition successfully. The HuskyLens AI Camera was able to detect the trained object and send the detected ID to the Arduino Uno. Based on the detection result, the Arduino successfully controlled the LED output correctly.

- The LED turned ON when object 1 detected.
- The LED turned OFF when no object was shown to the camera.

So, from this , it can be proved that the HuskyLens can work with Arduino for simple AI object detection and LED control applications.

Task 2- Multi-Color Recognition

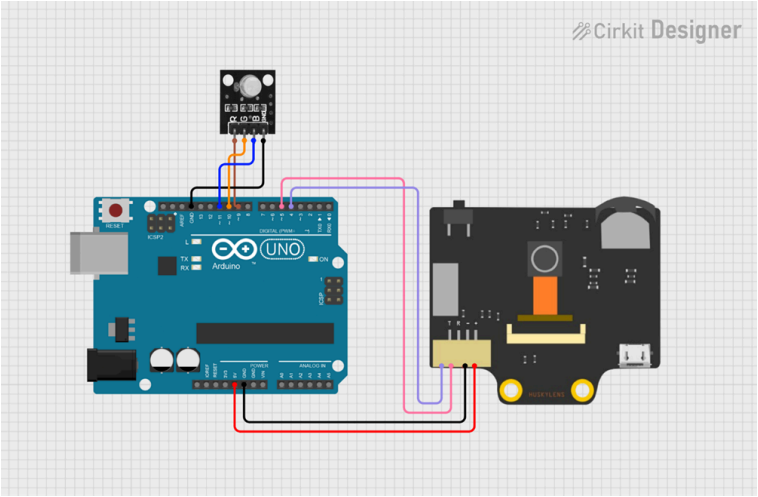
2. Material and Equipment

Cytron CT Uno
USB Cable
Gravity HuskyLens AI Vision Camera
RGB LED
Jumper wires

3. Experimental Setup

RGB LED to Arduino connection:
1. Ground to Ground
2. R to D9
3. G to D10
4. B to D11

Husky-Lens to Arduino connection:
1. VCC to Arduino 5V
2. GND to Arduino GND
3. TX to Arduino D4 (SoftwareSerial RX)
4. RX to Arduino D5 (SoftwareSerial TX)



Circuit Diagram Connection for Multi Color Recognition

4. Methodology

Hardware Setup

The initial phase of the experiment involves the physical integration of the HuskyLens AI Vision Camera and the HW-479 RGB LED module with the Arduino Uno. The system uses a SoftwareSerial interface on digital pins D4 and D5 to guarantee that the main components serial port is still accessible for PC debugging. The Arduino's 5V and GND pins power the HuskyLens, and its communication lines are "crossed" to enable data exchange: the camera's TX pin is linked to the Arduino's D4 (RX), while its RX pin is connected to D5 (TX). In order to enable color mixing, the RGB LED module is integrated together with PWM-capable pins. The circuit is completed by connecting the module's Red, Green, and Blue terminals to Arduino pins D9, D10, and D11, respectively.

Color Recognition

The AI vision sensor is manually trained in the next phase to recognize particular colors. The "Color Recognition" mode is selected by the user by flipping the HuskyLens function wheel. A red object is shown to the camera, and when the "Learn" button is hit to store it as ID 1, the training proceeds in a consecutive order. For a blue object (ID 2) and a green object (ID 3), the same procedure is repeated. Other colors, such as purple and yellow, can be trained as ID 4 and ID 5 for enhanced integration. Because the sensor records precise pixel data that might be distorted by shadows or overexposure, it is critical to maintain constant lighting during this operation.

Programming and Implementation

After training, the Arduino program was uploaded to establish communication with the HuskyLens. The system continuously requested data from the camera and processed the detected object IDs in real time. Based on the received ID, corresponding actions were executed to control the RGB LED output. The programming was implemented using Arduino IDE with the integration of the HuskyLens library and SoftwareSerial library.

Arduino Coding Code:

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX
```

```
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  Serial.begin(115200);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  setColor(0, 0, 0); // LED OFF

  while (!huskylens.begin(mySerial)) {
    Serial.println(F("Begin failed! Check wiring and Protocol Serial 9600"));
    delay(500);
  }

  Serial.println("HuskyLens Color Recognition Ready!");
}

void loop() {
  bool colorDetected = false;

  if (!huskylens.request()) {
    setColor(0, 0, 0);
    return;
  }

  while (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    if (result.command == COMMAND_RETURN_BLOCK) {
      colorDetected = true;

      switch (result.ID) {
        case 1: // Red
          Serial.println("RED detected");
```

```
        setColor(255, 0, 0);
        break;

      case 2: // Blue
        Serial.println("BLUE detected");
        setColor(0, 0, 255);
        break;

      case 3: // Green
        Serial.println("GREEN detected");
        setColor(0, 255, 0);
        break;

      default:
        Serial.println("Unknown color detected");
        setColor(0, 0, 0);
        break;
    }
  }
}

if (colorDetected == false) {
  setColor(0, 0, 0);
}
}

void setColor(int redValue, int greenValue, int blueValue) {
  analogWrite(redPin, redValue);
  analogWrite(greenPin, greenValue);
  analogWrite(bluePin, blueValue);
}
```

Testing

Actual-time performance tests of the integrated system are part of the last validation stage. To ensure that the RGB LED begins an appropriate color match with minimal delay, each trained color is shown to the HuskyLens. The lighting conditions may have changed since the initial training phase if the LED displays the incorrect color or the camera is unable to identify a trained object.

5. Data Collection

In this experiment, a total of three trials were carried out using three different colors, namely red, blue, and green. Each color was trained in the HuskyLens under Color Recognition mode and assigned a unique ID.

For each trial, a colored object was placed in front of the HuskyLens camera at a consistent distance. The system was allowed a short time to stabilize before observations were recorded. The detected ID from the HuskyLens was monitored through the Arduino Serial Monitor, while the corresponding LED output was verified visually using the RGB LED module.

To maintain consistency and accuracy, all trials were performed under stable indoor lighting conditions. This was important to ensure that the camera could reliably recognize the trained colors without interference from shadows or brightness variations.

The results showed that the system was able to correctly detect all three trained colors and produce the corresponding LED output. The RGB LED successfully displayed red, blue, and green when the respective colors were presented to the camera. This indicates that the system is functioning as expected under controlled conditions.

Overall, the data collection shows that, under the right training and ambient conditions, the HuskyLens sensor and Arduino combination works well for real-time color detection.

6. Results

Trial	Color	Detect ID	LED Output
1	Red	1	Red
2	Blue	2	Blue
3	Green	3	Green

7. Data Analysis

Based on the results obtained, the system achieved **100% accuracy** in detecting the trained colors (red, blue, and green). Each color was consistently identified with the correct ID, and the RGB LED responded accordingly without delay.

However, it is important to note that this level of accuracy was achieved under controlled conditions. Factors such as lighting intensity, object distance, and viewing angle can influence the performance of the HuskyLens. Although no errors were observed in this experiment, variations in environmental conditions may lead to incorrect detection in real-world applications.

In conclusion, the data analysis confirms that the system is highly effective for basic color detection tasks and successfully meets the objectives of the experiment.

Discussion

In this experiment, we evaluate the performance of the HuskyLens AI vision sensor integrated with Arduino in both colour detection and object detection modes. In Task 1, HuskyLens supposedly detected the trained objects while in Task 2, specific colours need to be identified. The ID detected was transmitted to the Arduino to be processed and given command to control the output. Mainly, this experiment was to understand the AI vision application in embedded systems.

In Task 1, HuskyLens successfully recognized the trained object that had been stored in its memory. The system was able to distinguish trained objects and provide object ID and positional data to Arduino. The arduino successfully controlled the output, which is to turn ON the LED based on the ID sent by HuskyLens. However, it required more accurate training as it was more sensitive to changes in object angle, orientation and movement speed.

In Task 2, HuskyLens was able to identify specific colors that have been stored in its memory. When the colours detected, the ID stored was sent to the Arduino to be processed and turned ON the RGB LED according to the detected colour. The results showed that colour detection generally responded quickly and accurately when the target colour was distinct and lighting conditions were stable. But, discrepancies were observed when lighting conditions changed or when similar background colours interfered with detection where HuskyLens occasionally misidentified colours or produced unstable tracking.

Yet, several sources of error may have influenced the experiment like environmental and insufficient training. Plus, HuskyLens itself has limited processing capability compared to advanced computer vision systems, making it less effective in highly dynamic or complex

environments. Despite these limitations, the experiment has important implications. It demonstrates that HuskyLens is capable of performing both colour-based and object-based AI detection in a practical and accessible way.

Conclusion

In conclusion, the HuskyLens AI Vision and Real-Time Color Detection experiment successfully demonstrated how AI-based visual recognition can be integrated with Arduino for interactive hardware control. The system successfully identified trained objects and colours accurately, then used the detected IDs to control the output which in this experiment was LED and RGB LED. Even so, external factors such as ambient lighting can affect recognition accuracy, making proper environmental setup and sensor retraining essential for consistent results. Overall, this project provided valuable insight into the application of AI vision technology in real-world automation systems, robotics, and smart device development.

Recommendation

The source of error can be reduced by improving a few things in this experiment. First, lighting conditions need to be improved where the experiment needs to be conducted in a controlled lighting environment to reduce errors caused by shadows, reflections, or brightness changes. Consistent lighting improves HuskyLens color recognition accuracy. Then, HuskyLens need to be trained for different environments to ensure more reliable detection results. Lastly, use stable hardware connections especially TX/RX communication lines and LED output pins, to prevent communication failures or incorrect LED responses.

References

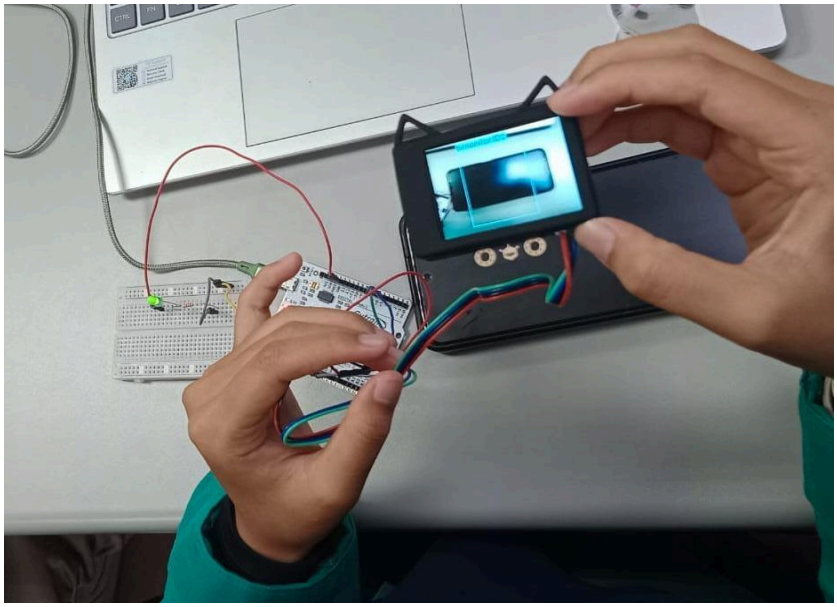
1. AI in Automation / Robotics
   S. Russell & P. Norvig, Artificial Intelligence: A Modern Approach, Pearson.

2. https://github.com/HuskyLens/HUSKYLENSArduino

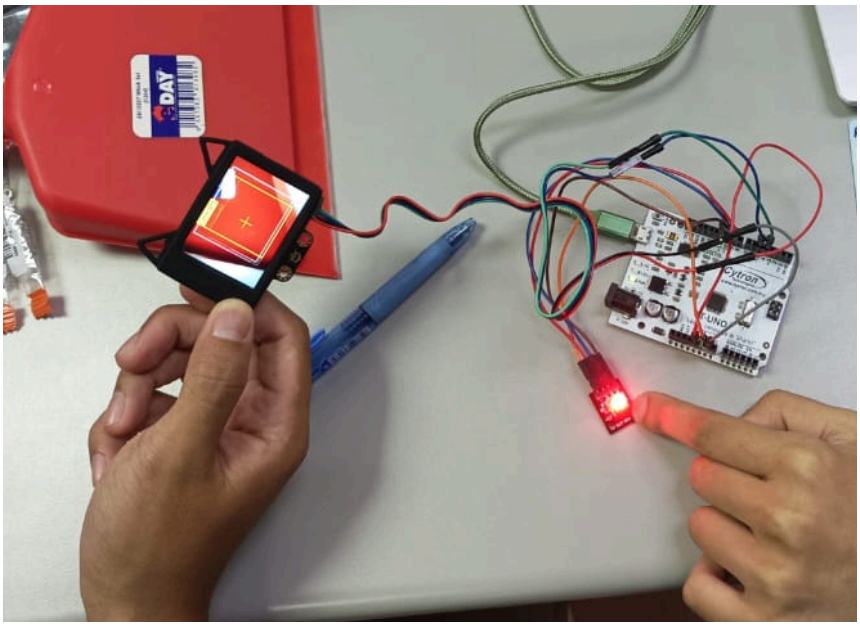
3. DFRobot. *HuskyLens AI Camera User Manual*. Available at: https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336

4. Arduino. “SoftwareSerial Library.” Available at: https://docs.arduino.cc/learn/built-in-libraries/software-serial

Appendices



Appendices 1: Circuit Setup for Object Recognition Control



Appendices 2: Circuit Setup for Multi-Color Recognition

Appendices 3: Arduino Coding for Object Recognition Control

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

const int ledPin = 13;

void setup() {
  Serial.begin(115200);
  mySerial.begin(9600);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  while (!huskylens.begin(mySerial)) {
    Serial.println(F("Begin failed! Check wiring and Protocol Serial 9600"));
    delay(500);
  }

  Serial.println("HuskyLens Ready!");
}

void loop() {
  bool objectDetected = false;

  if (!huskylens.request()) {
    digitalWrite(ledPin, LOW);
    return;
  }

  while (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    if (result.command == COMMAND_RETURN_BLOCK) {
      Serial.print("Object ");
```

```
      Serial.print(result.ID);
      Serial.println(" detected!");

      if (result.ID == 1) {
        objectDetected = true;
      }
    }
  }

  if (objectDetected == true) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
```

Appendices 4: Arduino Coding for Multi-Color Recognition

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  Serial.begin(115200);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  setColor(0, 0, 0); // LED OFF

  while (!huskylens.begin(mySerial)) {
```

```
    Serial.println(F("Begin failed! Check wiring and Protocol Serial 9600"));
    delay(500);
  }

  Serial.println("HuskyLens Color Recognition Ready!");
}

void loop() {
  bool colorDetected = false;

  if (!huskylens.request()) {
    setColor(0, 0, 0);
    return;
  }

  while (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    if (result.command == COMMAND_RETURN_BLOCK) {
      colorDetected = true;

      switch (result.ID) {
        case 1: // Red
          Serial.println("RED detected");
          setColor(255, 0, 0);
          break;

        case 2: // Blue
          Serial.println("BLUE detected");
          setColor(0, 0, 255);
          break;

        case 3: // Green
          Serial.println("GREEN detected");
          setColor(0, 255, 0);
          break;

        case 4: // Yellow
          Serial.println("YELLOW detected");
          setColor(255, 255, 0);
```

```
        break;

      case 5: // Purple
        Serial.println("PURPLE detected");
        setColor(255, 0, 255);
        break;

      default:
        Serial.println("Unknown color detected");
        setColor(0, 0, 0);
        break;
    }
  }
}

if (colorDetected == false) {
  setColor(0, 0, 0);
}
}

void setColor(int redValue, int greenValue, int blueValue) {
  analogWrite(redPin, redValue);
  analogWrite(greenPin, greenValue);
  analogWrite(bluePin, blueValue);
}
```

Acknowledgement

First of all, we would like to sincerely thank our lecturer, Dr.-Ing. Wahju Sediono, for his guidance and support throughout this experiment. His explanations and advice really helped us understand how the HuskyLens AI Vision works, the communication between HuskyLens and Arduino in controlling the output, and intelligent control systems more clearly.

We would also like to thank all our group members for their cooperation and teamwork. Everyone contributed their ideas and effort, which made it easier for us to complete all the tasks and overcome the challenges during the lab sessions.

Besides that, we are grateful to our classmates for their help and discussions, which made the learning process more enjoyable and meaningful.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report.** The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us.**

Signature: Read [/]
Name: Nur Irdina Binti Omar Understand [/]
Matric Number: 2312378 Agree [/]
Contribution: Task 1: Material and Equipment, Methodology, Experimental Setup, Data Collection, Data Analysis, Result.

Signature: Read [/]
Name: Muhammad Irsyad Ilham Bin Azizan Understand [/]
Matric Number: 2217555 Agree [/]
Contribution: Task 2: Material and Equipment, Methodology, Experimental Setup, Data Collection, Data Analysis, Result.

Signature: Read [/]
Name: Nurin Najwa binti Kamaruzaman Understand [/]
Matric Number: 2416554 Agree [/]
Contribution: Abstract, Introduction, Discussion, Conclusion, Recommendation, References, Appendices and Acknowledgement.